

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ

Циклова комісія інформаційних технологій

Звіт з практичних робіт
З дисципліни
Мікропроцесорні системи

Студента _3_ курсу __ групи
спеціальності : 123 Комп'ютерна
інженерія

Освітньо-професійна програма: 5.123.1
Обслуговування комп'ютерних систем і
мереж

Молоткова А.С.

Керівник: Дігтяр Артем Андрійович
Національна
шкала _____

Кількість балів: _____ Оцінка: ECTS

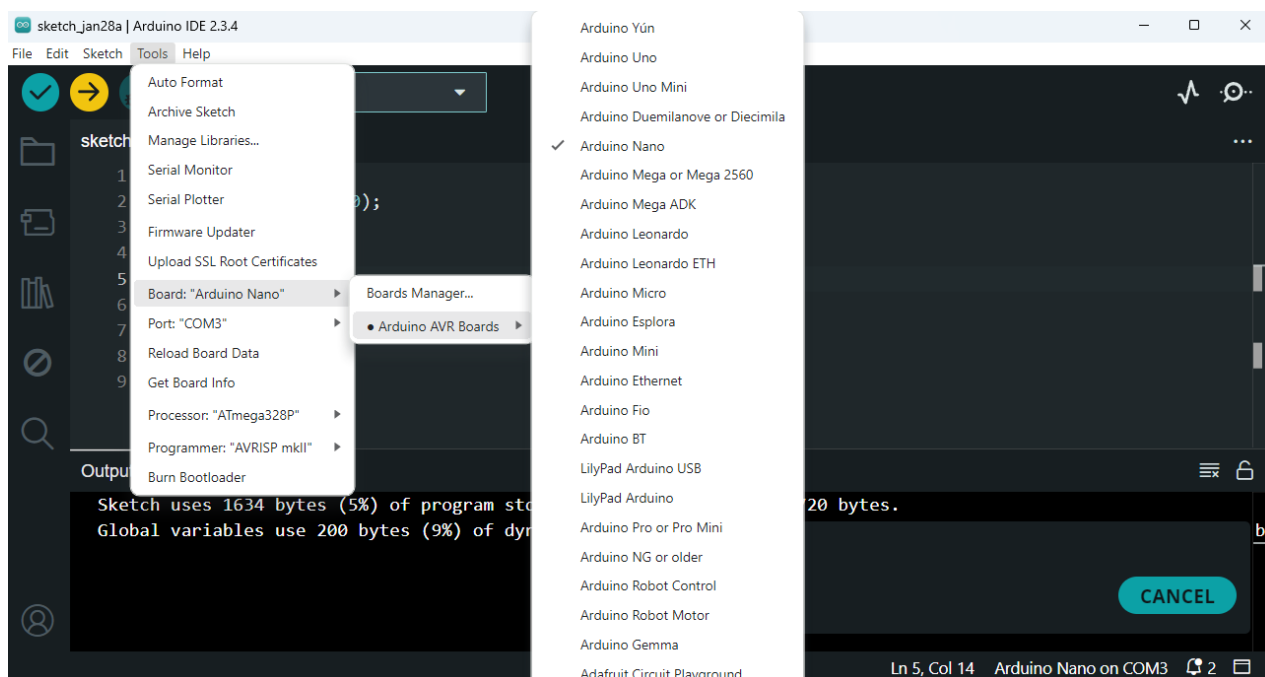
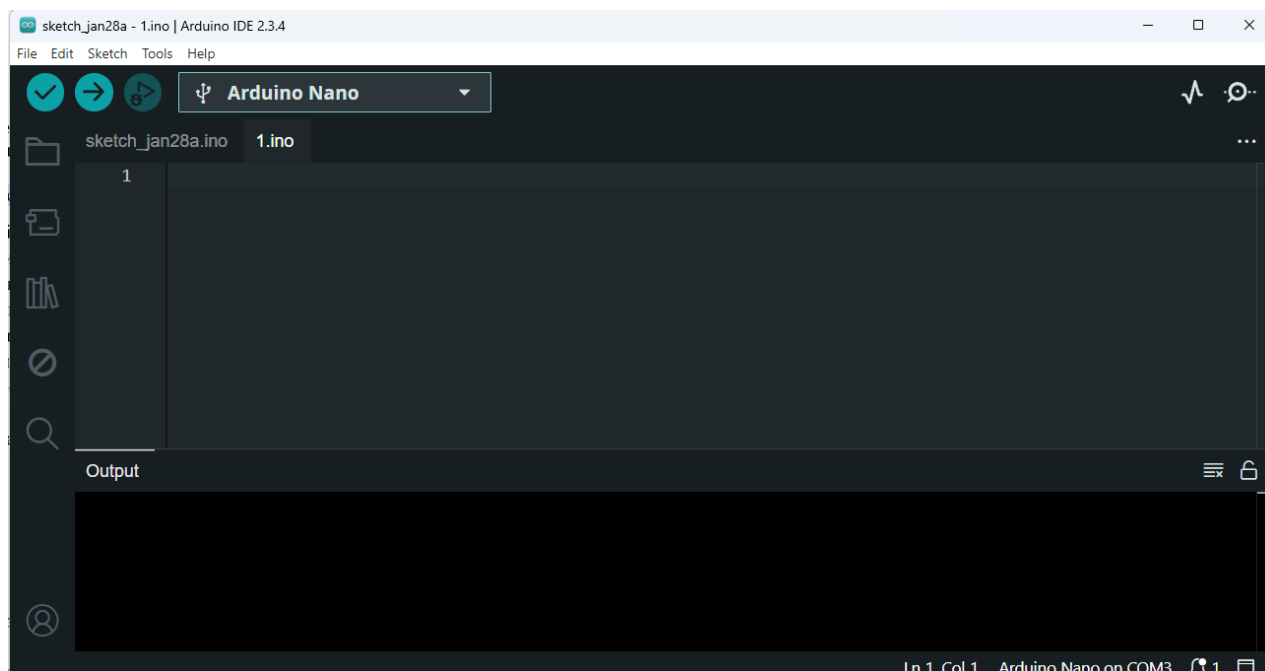
Лабораторна робота №1.

Тема: Ознайомлення з інтегрованою середою програмування мікроконтролерів. Дослідження роботи з портами

Мета роботи: Ознайомитися з інтегрованою середою програмування на ПК

Хід роботи:

Arduino IDE:



					5.123.1.28.04-ЛР				
Змін.	Арк.	№ докум.	Підпис	Дата	“Ознайомитися з інтегрованою середою програмування на ПК”	Лім.	Арк.	Аркушів	
Розроб.		Молотков А.		28.01					
Перевір.		Дігтяр А.				у	1	2	
Т. контр.						ХПФК група 37-кі			
Н. Контр.									
Затверд.									

Контрольні питання:

1. Arduino — платформа мікроконтролерів, плат та середовища розробки для створення електронних пристроїв.
2. Arduino IDE — офіційне програмне середовище для написання, компіляції та завантаження програм у пам'ять мікроконтролера на платах Arduino, що використовує мову Processing/Wiring: похідна C++. Сердовище працює на ОС Windows, Linux та Mac OS.
3. Апаратна частина Arduino складається з мікроконтролера, цифрових і аналогових виводів, USB-порту, роз'ємів живлення, світлодіодів, кварцового резонатора та додаткових чіпів.
 - a. Мікроконтролер — основний компонент, який виконує програмний код.
 - b. Цифрові й аналогові виводи — використовуються для підключення датчиків, модулів і пристроїв.
 - c. USB-порт — забезпечує передачу даних та живлення від комп'ютера.
 - d. Роз'єми живлення — дозволяють підключати зовнішнє джерело енергії.
 - e. Світлодіоди — індикатори роботи, наприклад, для відображення живлення чи передачі даних.
 - f. Кварцовий резонатор — задає точність таймінгів для мікроконтролера.
 - g. Додаткові чіпи — наприклад, USB-контролери серії CH340 для обміну даними з комп'ютером.

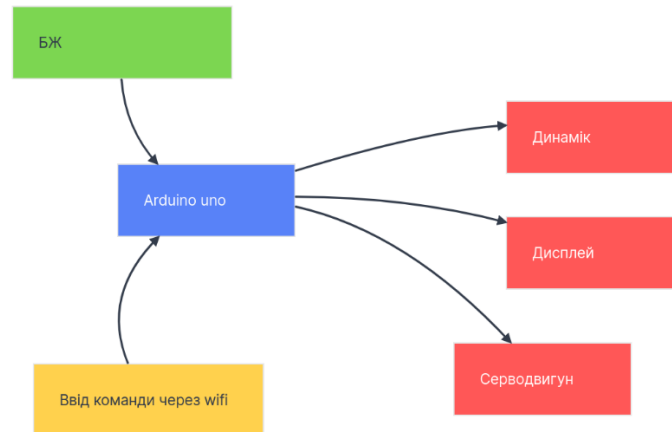
Висновок: я досяг мети та вивчив мету лр, а також виконав інструкції та відповів на контрольні питання.

Лабораторна робота №2. Розробка схеми електричної принципової мікропроцесорної системи.

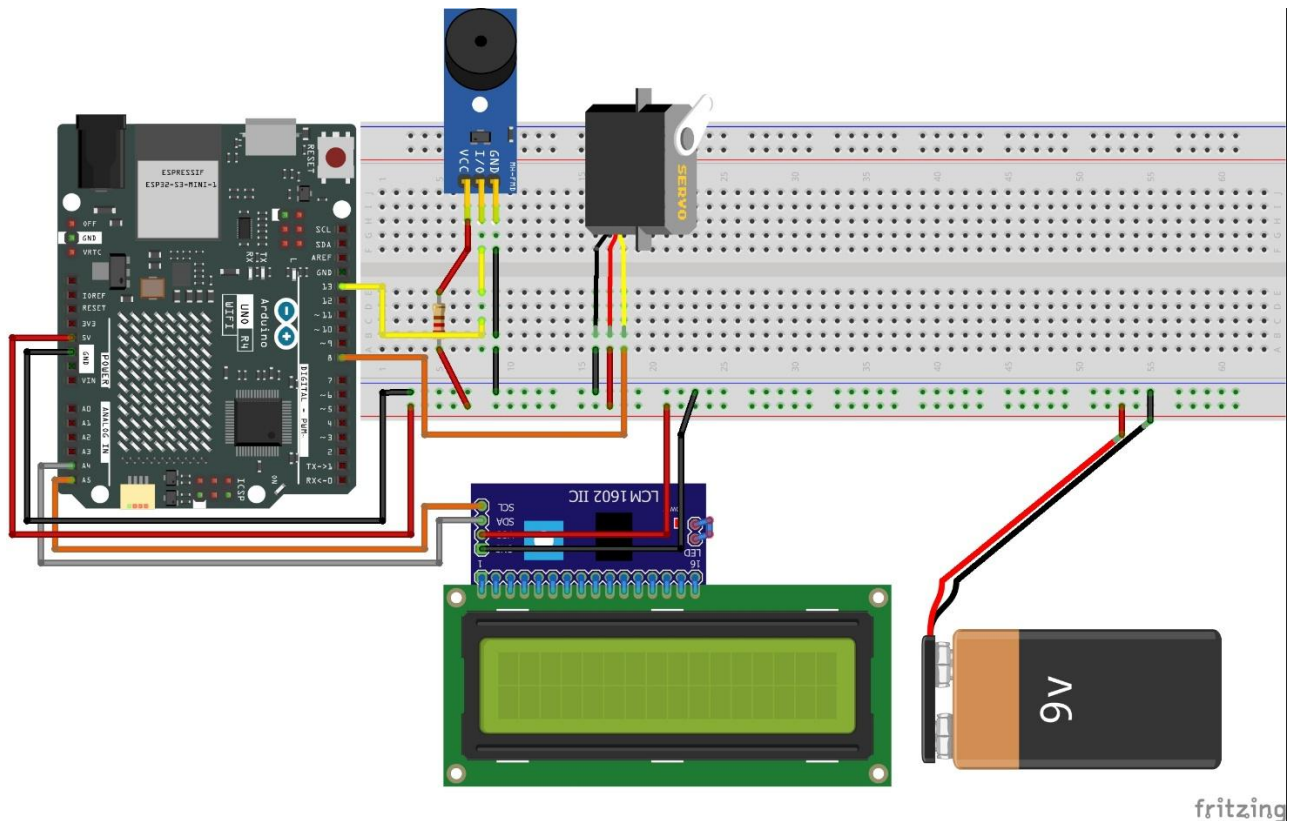
Мета роботи: ознайомитись з редактором схем САПР Fritzing, налаштуванням його опцій, навчитись підключати бібліотеки компонентів, створювати електричні принципові схеми.

Хід роботи:

Структурна схема:

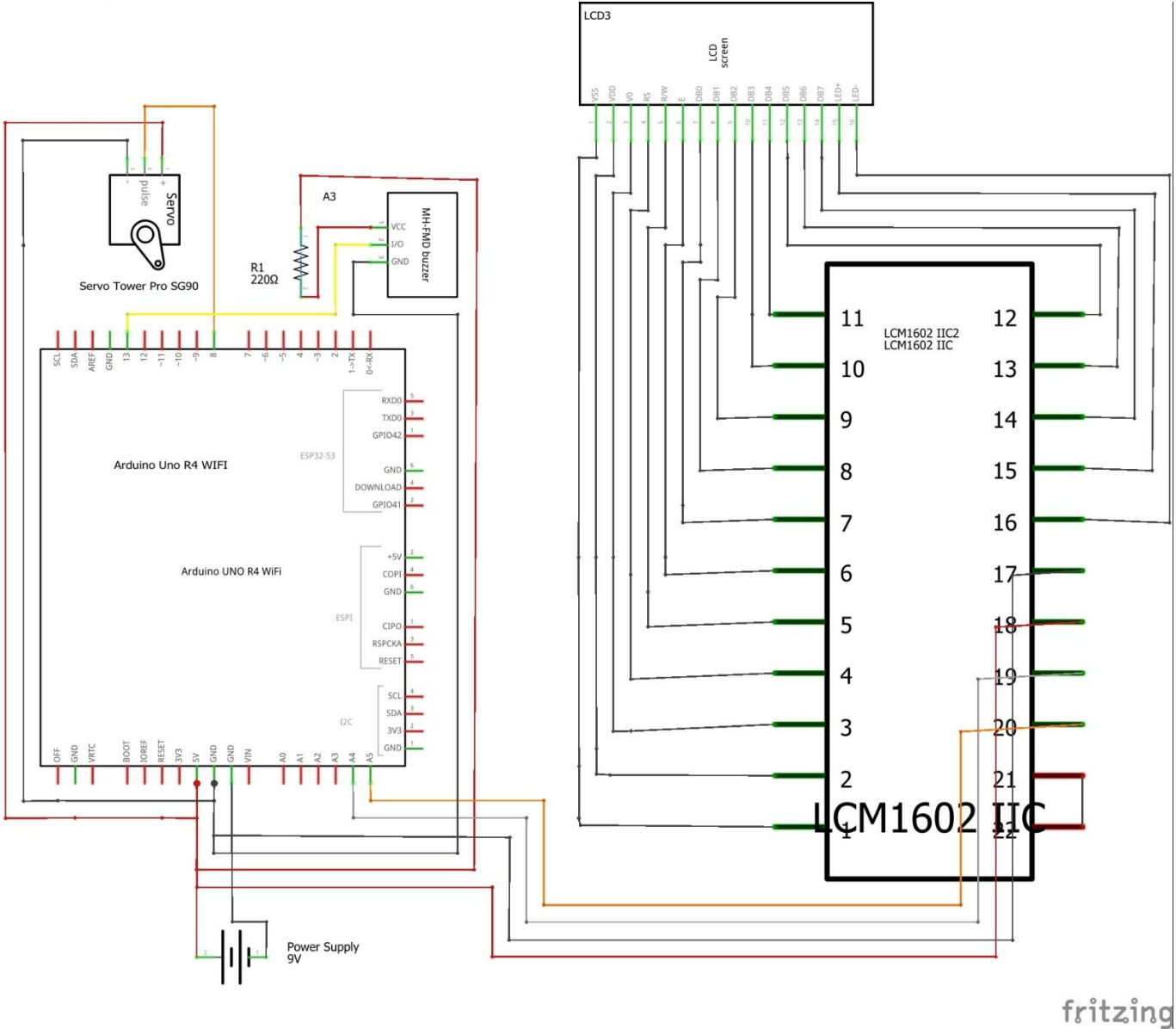


Схеми з CAD Fritzing: 1.Мотажна плата:

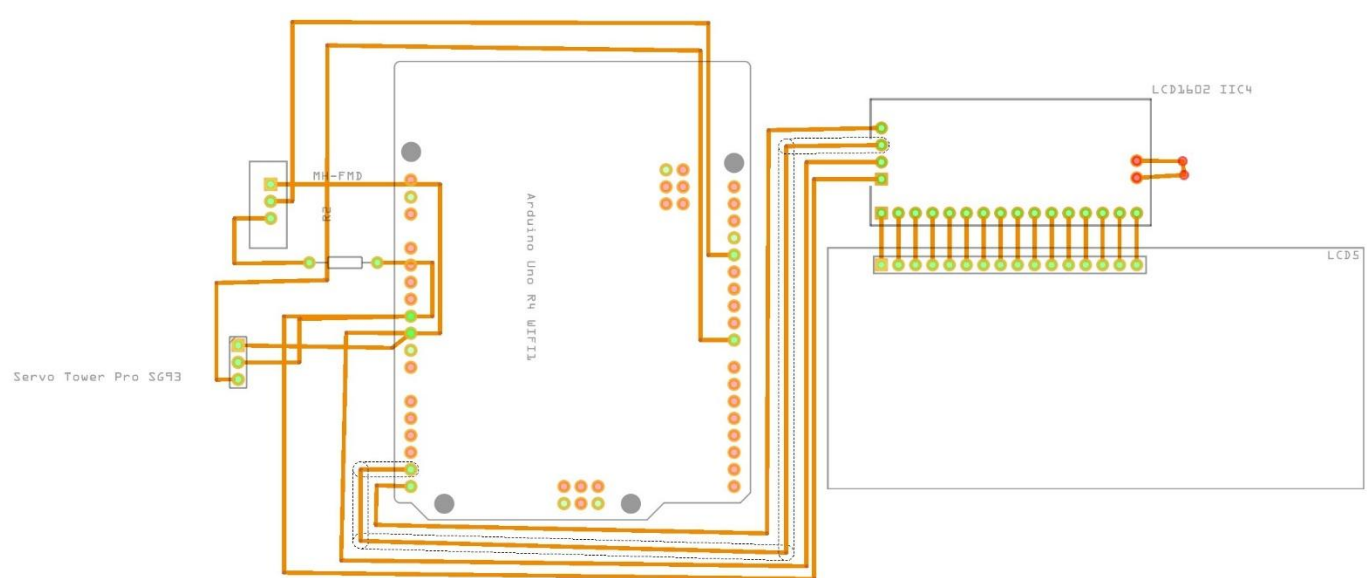


					5.123.1.28.04-ЛР				
Змін.	Арк.	№ докум.	Підпис	Дата	“Ознайомитись з редактором схем САПР Fritzing, налаштуванням його опцій, навчитись підключати бібліотеки компонентів, створювати електричні принципові схеми”.	Лім.	Арк.	Аркушів	
Розроб.		Молотков А.		03.04		У	1	2	
Перевір.		Дігтяр А.				ХПФК група 37-кі			
Т. контр.									
Н. Контр.									
Затверд.									

2.Електронна схема:



3. Друкована плата:



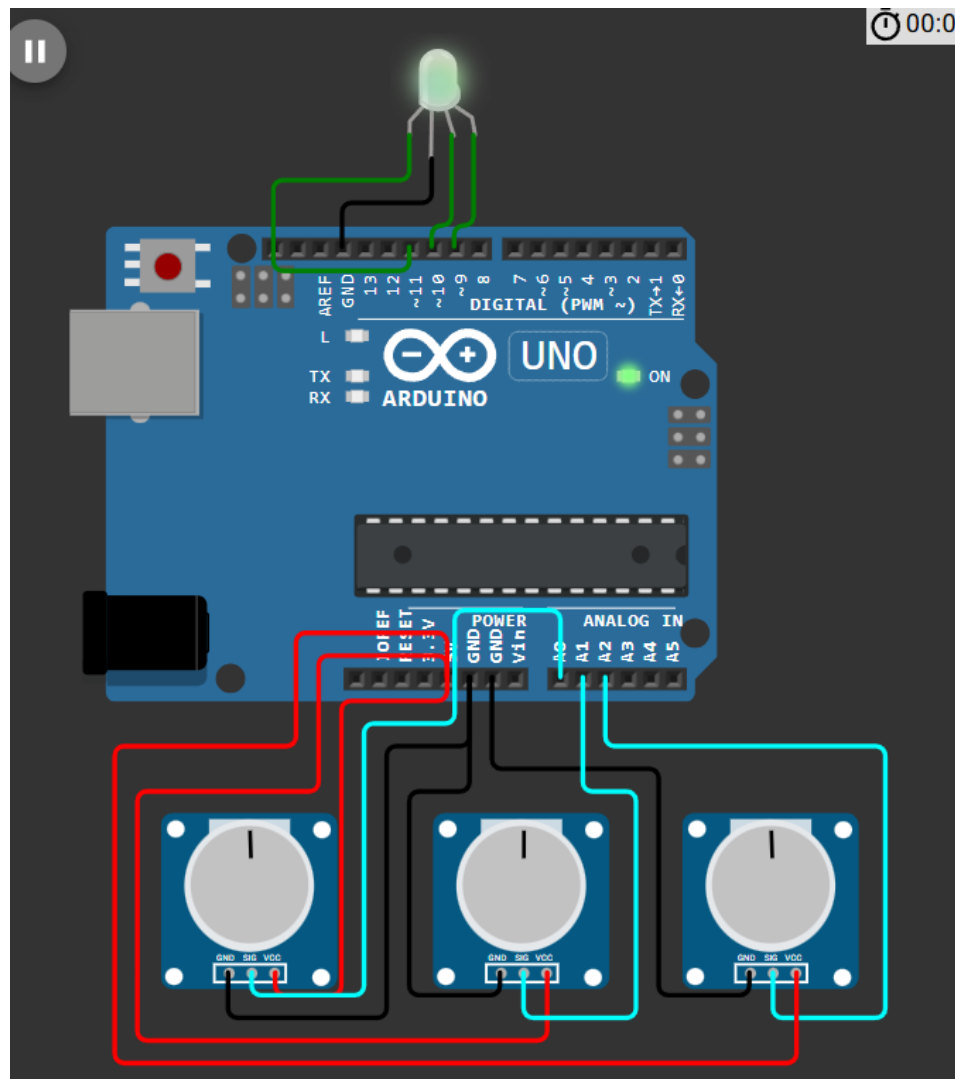
Лабораторна робота №3

Робота з аналоговими входами та потенціометром.

Мета роботи: навчитися працювати з аналоговими входами та генерувати ШІМ-сигнали за допомогою плати Arduino. Зокрема, лабораторна робота буде присвячена використанню кількох потенціометрів для керування кольором RGB-світлодіода за допомогою генерації ШІМ-сигналів. Для цього необхідно навчитися зчитувати аналогові значення з потенціометра за допомогою аналогових виводів плати Arduino та використовувати це значення для генерації ШІМ-сигналу для керування яскравістю світлодіода. Наприкінці лабораторної роботи студенти повинні добре розуміти, як використовувати аналогові виводи та генерувати ШІМ-сигнали у своїх проектах з Arduino.

Хід роботи :

Скрін роботи схеми:



					5.123.1.17.03-ЛР				
Змін.	Арк.	№ докум.	Підпис	Дата	“Навчитися працювати з аналоговими входами та генерувати ШІМ-сигнали за допомогою плати Arduino”.		Літ.	Арк.	Аркушів
Розроб.		Молотков А.		27.05			у	1	3
Перевір.		Дігтяр А.					ХДПК група 37-кі		
Т. контр.									
Н. Контр.									
Затверд.									

Скетч:

```
const int redLedPin = 11;
const int greenLedPin = 10;
const int blueLedPin = 9;
void setup() {
  pinMode(A0, INPUT);
  pinMode(A1, INPUT);
  pinMode(A2, INPUT);
  pinMode(redLedPin, OUTPUT);
  pinMode(greenLedPin, OUTPUT);
  pinMode(blueLedPin, OUTPUT);
}

void loop() {
  int red = analogRead(A0);
  int green = analogRead(A1);
  int blue = analogRead(A2);

  red = map(red, 0, 1023, 0, 255);
  green = map(green, 0, 1023, 0, 255);
  blue = map(blue, 0, 1023, 0, 255);

  analogWrite(9, blue);
  analogWrite(10, green);
  analogWrite(11, red);
}
```

Контрольні питання:

1. Аналоговий сигнал - це сигнал, який може мати діапазон значень, на відміну від цифрового сигналу, який може мати лише два значення (HIGH або LOW).
2. ШІМ (широтно-імпульсна модуляція) - це метод, що використовується в електроніці для керування потужністю, яка подається на навантаження, шляхом керування шириною вихідного сигналу. ШІМ-сигнали є цифровими сигналами, які можуть використовуватися для імітації аналогового сигналу, змінюючи шпаруватість сигналу.
3. Потенціометр використовується для зчитування аналогових значень напруги. У даній лабораторній роботі потенціометри використовуються для керування кольором RGB-світлодіода шляхом зміни аналогових значень, які потім

					5.123.1.17.03-ЛР	Лист
						2
Изм.	Лист	№ докум.	Подпись	Дата		

перетворюються на ШІМ-сигнали для керування яскравістю кожного кольорового каналу світлодіода.

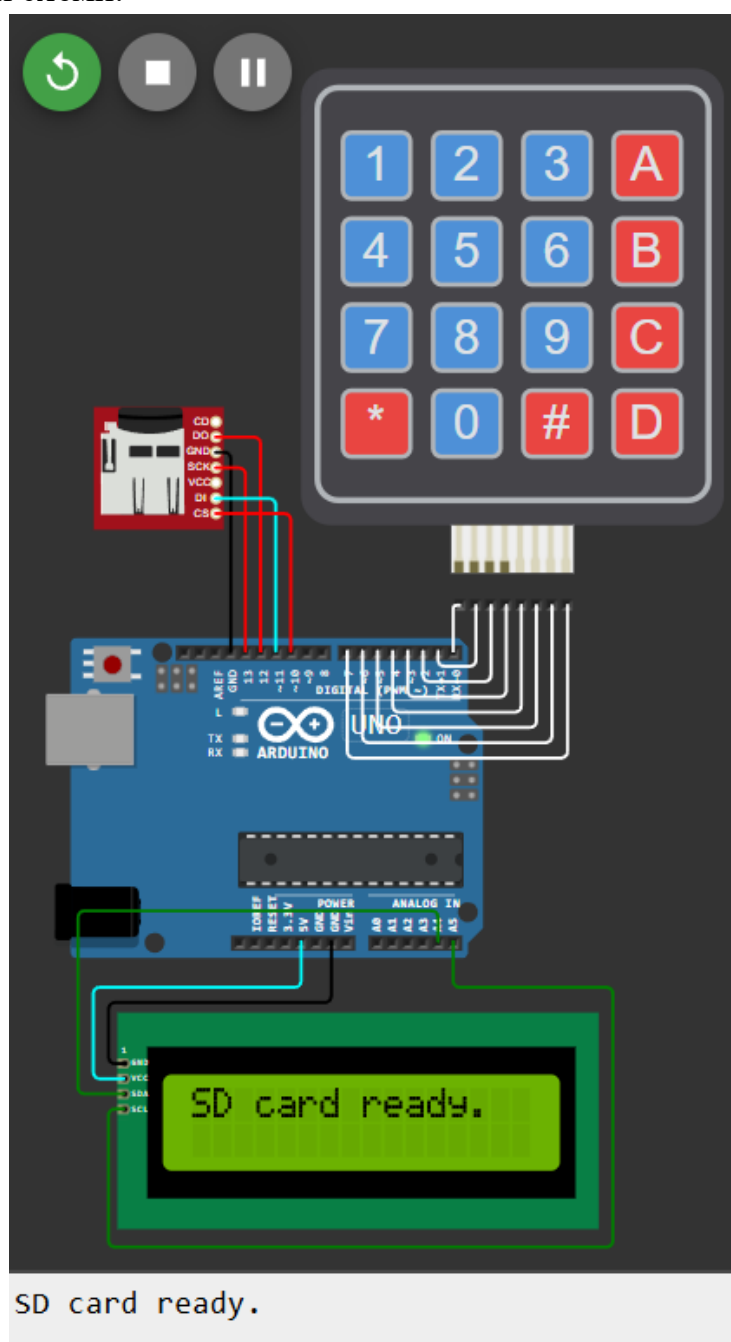
Висновок: я вивчив тему та досяг мети лабораторної роботи, та відповів на контрольні питання.

Лабораторна робота №4 Комунікація I2C та UART.

Мета роботи: знайомство з принципами роботи протоколів зв'язку I2C, SPI та UART, навчання використанню цих протоколів в Arduino, а також розвиток практичних навичок реалізації зв'язку між різними пристроями, що використовують комунікацію I2C, SPI та UART.

Хід роботи :

Скрін роботи схеми:



					5.123.1.17.04-ЛР				
Змін.	Арк.	№ докум.	Підпис	Дата					
Розроб.		Молотков А.		27.05	“Знайомство з принципами роботи протоколів зв'язку I2C, SPI та UART, навчання використанню цих протоколів в Arduino, а також розвиток практичних навичок реалізації зв'язку між різними пристроями, що використовують комунікацію I2C, SPI та UART”.	Лім.	Арк.	Аркушів	
Перевір.		Дігтяр А.				у	1	5	
Т. контр.						ХДПК група 37-кі			
Н. Контр.									
Затверд.									

Пояснення роботи:

1. Підготовка та додавання бібліотек у Wokwi

Перед початком роботи з кодом у Wokwi.com вам необхідно додати необхідні бібліотеки, оскільки вони не завантажуються автоматично. Для цього в інтерфейсі Wokwi зліва, де знаходиться код, ви знайдете вкладку "libraries.txt". Відкривши її, вам потрібно буде додати кожен бібліотеку з рядка #include окремим рядком у цей файл. Для цього коду це будуть: Keypad, SD, LiquidCrystal_I2C. Просто введіть назву кожної бібліотеки в окремому рядку та збережіть файл. Wokwi автоматично завантажить їх, коли ви запусите симуляцію.

2. Глобальні визначення та змінні

Цей розділ коду відповідає за визначення констант та глобальних змінних, які будуть використовуватися протягом усієї програми. Це дозволяє легко налаштовувати піни підключення, адреси та розміри екрану, а також оголошувати об'єкти для роботи з периферійними пристроями, такими як SD-карта, клавіатура та LCD-дисплей.

```
#include <Keypad.h>
#include <SD.h>
#include <LiquidCrystal_I2C.h>

#define CS_PIN 10
#define I2C_ADDR 0x27
#define LCD_COLUMNS 20
#define LCD_LINES 4

File myFile;

const uint8_t ROWS = 4;
const uint8_t COLS = 4;

char keys[ROWS][COLS] = {
  { '1', '2', '3', 'A' },
  { '4', '5', '6', 'B' },
  { '7', '8', '9', 'C' },
  { '*', '0', '#', 'D' }
};

uint8_t colPins[COLS] = { 4, 5, 6, 7 };
uint8_t rowPins[ROWS] = { 0, 1, 2, 3 };

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
LiquidCrystal_I2C lcd(I2C_ADDR, LCD_COLUMNS, LCD_LINES);
```

3. Функція setup()

Функція setup() виконується один раз при запуску Arduino. Її основна мета – ініціалізувати всі необхідні пристрої та зв'язки. Тут відбувається налаштування послідовного порту для обміну даними з комп'ютером, ініціалізація LCD-дисплея та SD-карти. Якщо ініціалізація SD-карти завершиться невдачею, програма виведе повідомлення про помилку на послідовний порт та LCD, а потім зупиниться, щоб запобігти подальшій роботі з неініціалізованим пристроєм.

```
void setup() {  
    Serial.begin(9600);  
    lcd.init();  
    lcd.backlight();  
  
    if (!SD.begin(CS_PIN)) {  
        Serial.println("Card initialization failed!");  
        lcd.setCursor(0, 0);  
        lcd.print("SD init failed!");  
        while (true);  
    }  
  
    Serial.println("SD card ready.");  
    lcd.setCursor(0, 0);  
    lcd.print("SD card ready.");  
}
```

4. Функція loop(): Запис файлу

Функція loop() виконується безперервно після завершення setup(). Цей розділ коду відповідає за запис даних у файл test.txt на SD-карті. Коли користувач натискає клавішу #, програма відкриває файл для запису, виводить повідомлення про початок запису на послідовний порт та LCD. Дані, що надходять через послідовний порт, записуються у файл доти, доки користувач не натисне клавішу # знову. Після цього файл закривається, і на LCD відображається повідомлення про завершення запису.

```
void loop() {  
    char key = keypad.getKey();  
  
    if (key == '#') {  
        myFile = SD.open("test.txt", FILE_WRITE);  
        if (!myFile) {  
            Serial.println("Error opening file for write");  
            return;  
        }  
    }  
}
```

					5.123.1.17.04-ЛР	Лист
						3
Изм.	Лист	№ докум.	Подпись	Дата		

```

}

Serial.println("Start write file content:");
lcd.clear();
lcd.print("Writing...");

key = '0'; // dummy init
while (key != '#') {
    if (Serial.available()) {
        myFile.print(Serial.readString());
    }
    key = keypad.getKey();
}

Serial.println("End write file content:");
myFile.close();
lcd.clear();
lcd.print("Write done!");
delay(1000);
}
// ... (далі йде частина коду для читання файлу)
}

```

5. Функція loop(): Читання файлу

Ця частина функції loop() відповідає за читання вмісту файлу test.txt з SD-карти. Коли користувач натискає клавішу *, програма відкриває файл для читання. Вміст файлу послідовно зчитується посимвольно, виводиться на послідовний порт, а перші 16 символів відображаються на LCD-дисплеї. Після зчитування файл закривається, і відображається повідомлення про завершення операції.

```

void loop() {
    // ... (попередня частина коду для запису файлу)

    if (key == '*') {
        myFile = SD.open("test.txt");
        if (!myFile) {
            Serial.println("Error opening file for read");
            return;
        }

        Serial.println("Start file content:");
        lcd.clear();
        lcd.setCursor(0, 0);

        char buffer[17]; // 16 chars + null terminator
        int i = 0;
    }
}

```

```

while (myFile.available() && i < 16) {
    buffer[i] = myFile.read();
    Serial.write(buffer[i]);
    i++;
}
buffer[i] = '\0'; // null-terminate

lcd.print(buffer);
Serial.println();
Serial.println("End file content");

myFile.close();
delay(2000);
}
}

```

6. Створення файлу text.txt у Wokwi

Щоб код міг взаємодіяти з SD-картою та її файлами у симуляції Wokwi, вам потрібно створити файл text.txt вручну у вашому проєкті. У Wokwi.com, в панелі зліва, ви побачите вкладку "diagram.json". Поруч із нею є кнопка "+". Натисніть її, і у випадаючому меню виберіть "New File". Назвіть новий файл test.txt. Це створить порожній файл, який ваш скетч зможе використовувати для запису та читання даних. Це дозволить симулятору коректно емулювати роботу з SD-картою.

Висновок: я вивчив тему та досяг мети лабораторної роботи, та відповів на контрольні питання.

					5.123.1.17.04-ЛР	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

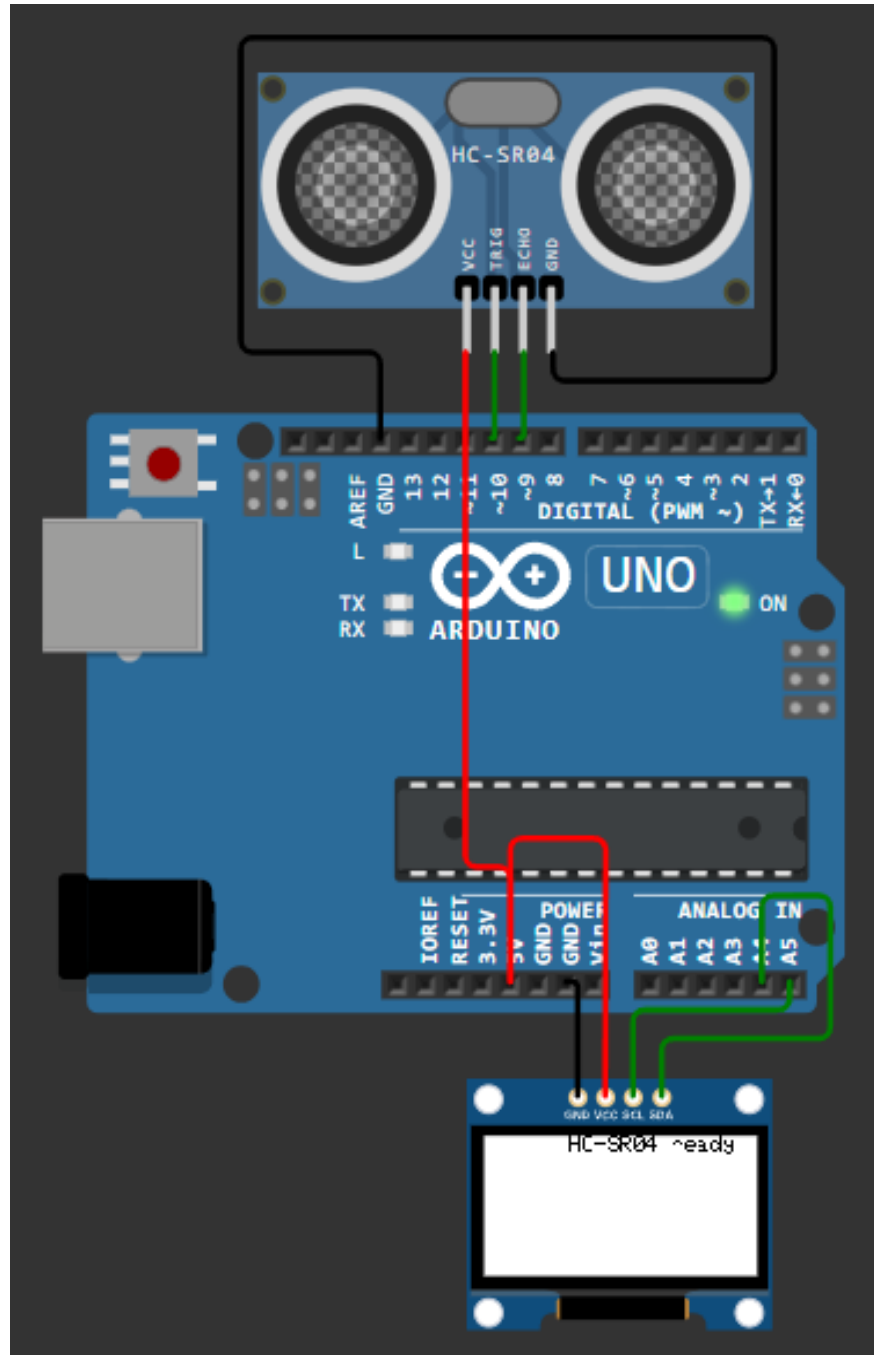
Лабораторна робота №5

Ультразвуковий датчик та OLED-дисплей

Мета: Ознайомитись з принципами роботи ультразвукового датчика HC-SR04 та OLED-дисплею з ардуіно, а також розробити простий проект, в якому відстань, виміряна датчиком, відображається на OLED-дисплеї.

Хід роботи :

Скрін роботи схеми:



					5.123.1.17.05-ЛР			
Змін.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Молотков А.		27.05	“Ознайомитись з принципами роботи ультразвукового датчика HC-SR04 та OLED-дисплею з ардуіно, а також розробити простий проект, в якому відстань, виміряна датчиком, відображається на OLED-дисплеї”.	Літ.	Арк.	Аркушів
Перевір.		Дігтяр А.				у	1	3
Т. контр.						ХДПК група 37-кі		
Н. Контр.								
Затверд.								

Скетч:

```
#include <NewPing.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define TRIGGER_PIN 10
#define ECHO_PIN 9
#define MAX_DISTANCE 200

NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);

#define OLED_RESET 4
Adafruit_SSD1306 display(128, 64, &Wire, OLED_RESET);

void setup() {
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    for(;;);
  }
  display.clearDisplay();

  display.fillScreen(WHITE); // фон білий
  display.setTextColor(BLACK); // текст чорний
  display.setTextSize(1);

  // Центруємо текст по горизонталі:
  int16_t x = (128 - (12 * 6)) / 2;
  display.setCursor(x, 0);
  display.print(" HC-SR04 ready");
  display.display();
  delay(5000);
}

void loop() {
  delay(200);
  unsigned int distance = sonar.ping_cm();

  display.fillScreen(WHITE);
  display.setTextColor(BLACK);

  display.setTextSize(1);
  display.setCursor(0, 0);
  display.print(" Distance:");

  display.setTextSize(2);
  display.setCursor(0, 20);

  if (distance > 0 && distance <= MAX_DISTANCE) {
    display.print(distance);
    display.print(" cm");
  } else {
    display.print("Out of range");
  }
}
```

					5.123.1.17.05-ЛР	Лист
						2
Изм.	Лист	№ докум.	Подпись	Дата		

```
display.display();
}
```

HC-SR04 працює за принципом ехолокації, схожим на сонар. Він випромінює ультразвуковий імпульс зі свого передавача (Trig-пін). Якщо імпульс зустрічає об'єкт, він відбивається і повертається до приймача (Echo-пін).

Ультразвуковий далекомір вимірює час між відправленням і отриманням сигналу. Знаючи швидкість звуку в повітрі (приблизно 343 м/с) та цей час, мікроконтролер Arduino Uno обчислює відстань. Оскільки звук проходить шлях туди й назад, виміряний час ділиться навпіл для визначення відстані до об'єкта. Це дозволяє досить точно вимірювати відстань до перешкод.

варіанти можливого використання ультразвукового:

- a. Робототехніка, зокрема підводні дрони: Допмагають роботам уникати перешкод та здійснювати автономну навігацію, виявляючи об'єкти та орієнтуючись під водою.
- b. Автотранспорт, Промисловість: Застосовуються для навігації у місті з низької видимості, вимірювання розмірів об'єктів на конвеєрах та для точного позиціонування у виробничих процесах.
- c. Системи безпеки та моніторингу: Використовуються як датчики присутності, наприклад, для сигналізацій, автоматичних дверей та для моніторингу рівня заповнення (рідин, сміття, зерна).
- d. Автоматизація та "розумний" дім: Забезпечують автоматичне освітлення та керування воротами/дверима при виявленні об'єктів або людей.
- e. Дослідницькі та освітні проекти: Використовуються для фізичних експериментів (вивчення швидкості звуку) та створення інтерактивних інсталяцій, що реагують на наближення.
- f.

Висновок: я вивчив тему та досяг мети лабораторної роботи, та відповів на контрольні питання.

					5.123.1.17.05-ЛР	Лист
						3
Изм.	Лист	№ докум.	Подпись	Дата		

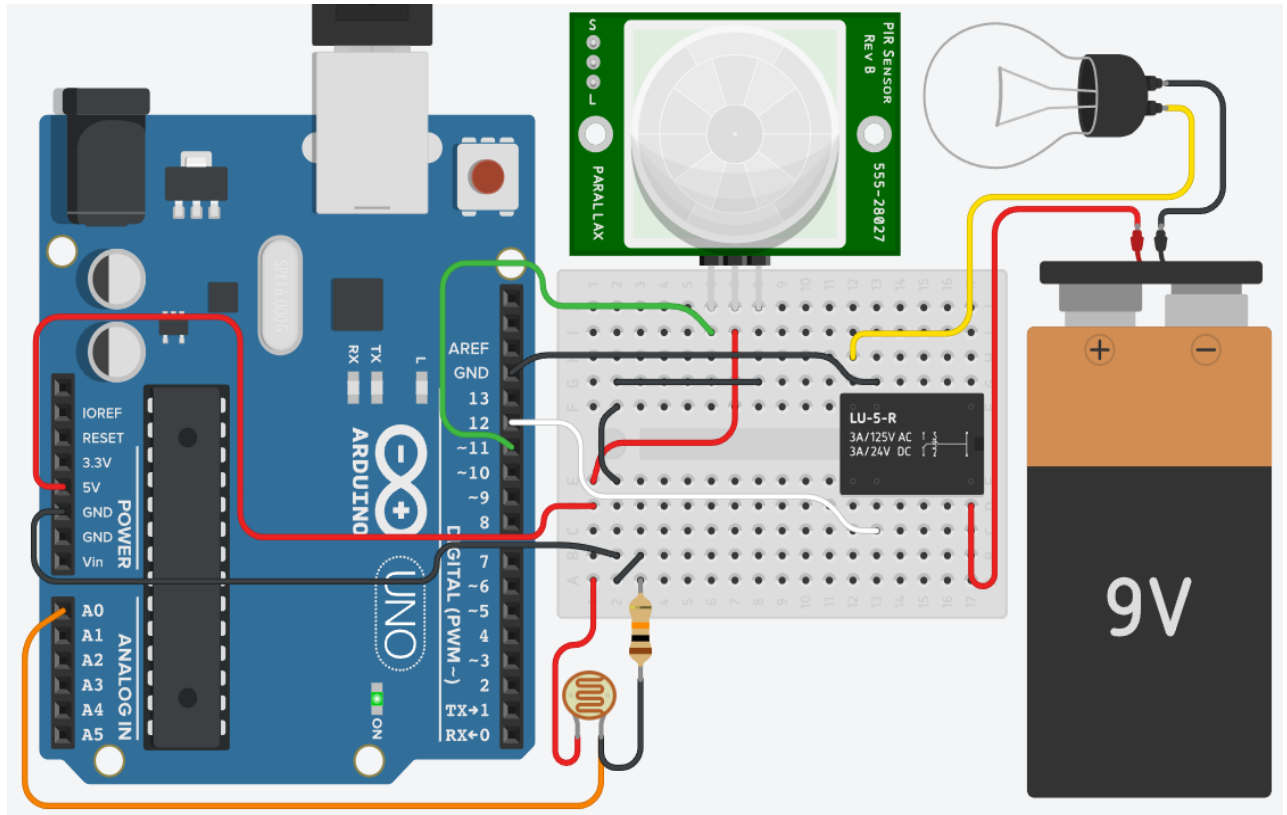
Лабораторна робота №6

Релейне керування

Мета роботи: навчитися проектувати та реалізовувати систему релейного управління освітленням приміщення, використовуючи Arduino, датчики освітленості та датчик рухів.

Хід роботи :

Скрін роботи схеми:



Скетч:

```
const int pirPin = 11;           // Сенсор
const int photoresistorPin = A1; // Фоторезистор
const int relayPin = 12;        // Реле

void setup() {
  pinMode(pirPin, INPUT);
  pinMode(relayPin, OUTPUT);
  digitalWrite(relayPin, LOW); // Реле вимкнуте
  Serial.begin(9600);
}
```

5.123.1.17.06-ЛР

Змін.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Молотков А.		27.05	“Навчитися проектувати та реалізовувати систему релейного управління освітленням приміщення, використовуючи Arduino, датчики освітленості та датчик рухів”.	Лім.	Арк.
Перевір.		Дігтяр А.				у	1
Т. контр.						ХДПК група 37-кі	
Н. Контр.							
Затверд.							
						Аркушів	3

```

void loop() {
  int pirState = digitalRead(pirPin);
  int lightValue = analogRead(photoresistorPin);

  //Вивід
  if (pirState == HIGH) {
    Serial.print("!! Movement! (");
    Serial.print(pirState);
    Serial.print("); Lighting: (");
    Serial.print(lightValue);
    Serial.println("); :0");
  } else {
    Serial.print("NO Movement (");
    Serial.print(pirState);
    Serial.print("); Lighting: (");
    Serial.print(lightValue);
    Serial.println("); :3");
  }

  // Сигналізація
  if (pirState == HIGH && lightValue < 600) {
    digitalWrite(relayPin, HIGH);
  } else {
    digitalWrite(relayPin, LOW);
  }

  delay(200);
}

```

Реле — це електро механічний перемикач, який використовує електромагніт для відкриття або закриття контактів, тим самим керуючи електричним колом. У своїй суті, це кнопка, яка активується електрикою. Коли на котушку реле подається невелика напруга, наприклад, від Arduino, вона створює магнітне поле, яке притягує металевий якір, замикаючи або розмикаючи контакти.

Використання реле в парі з мікроконтролерами, оскільки:

1. Гальванічна розв'язка: Реле забезпечує повну електричну ізоляцію між низьковольтним колом мікроконтролера (яке працює, наприклад, від 5В) та

					5.123.1.17.06-ЛР	Лист
						2
Изм.	Лист	№ докум.	Подпись	Дата		

високовольтним колом (наприклад, 220В змінного струму, що живить лампи). Це, у свою чергу, запобігає пошкодженню чутливої електроніки Arduino від високих напруг або стрибків струму, а також гарантує безпеку користувача.

2. Сумісність з високовольтними/високострумовими пристроями: Вони не можуть напряму вмикати або вимикати потужні пристрої, такі як освітлювальні прилади, двигуни чи опалювальні елементи, що працюють від мережі. Реле діє як інтерфейс, дозволяючи Arduino керувати великими навантаженнями, не перевантажуючи власні виводи.
3. Універсальність: Реле здатні керувати різними типами навантажень — як постійним (DC), так і змінним (AC) струмом, незалежно від їх напруги та струму (в межах допустимих для реле значень).
4. Надійність: Більшість реле є міцними та надійними компонентами, розрахованими на мільйони циклів перемикання, що забезпечує тривалу та стабільну роботу системи.

Висновок: я вивчив тему та досяг мети лабораторної роботи, відповів на контрольні питання, а також спроектував та реалізували систему автоматичного керування освітленням, яка реагує на рух та рівень освітленості. Цей проєкт дозволив нам глибоко зрозуміти принципи роботи реле, датчиків руху (PIR) та фоторезисторів у поєднанні з мікроконтролером Arduino.

Лабораторна робота №7

Підключення сервоприводу до arduino

Мета роботи: Навчитися складати схеми підключення сервоприводу до ARDUINO та складати програму управління.

Хід роботи:

Лістинг 2.

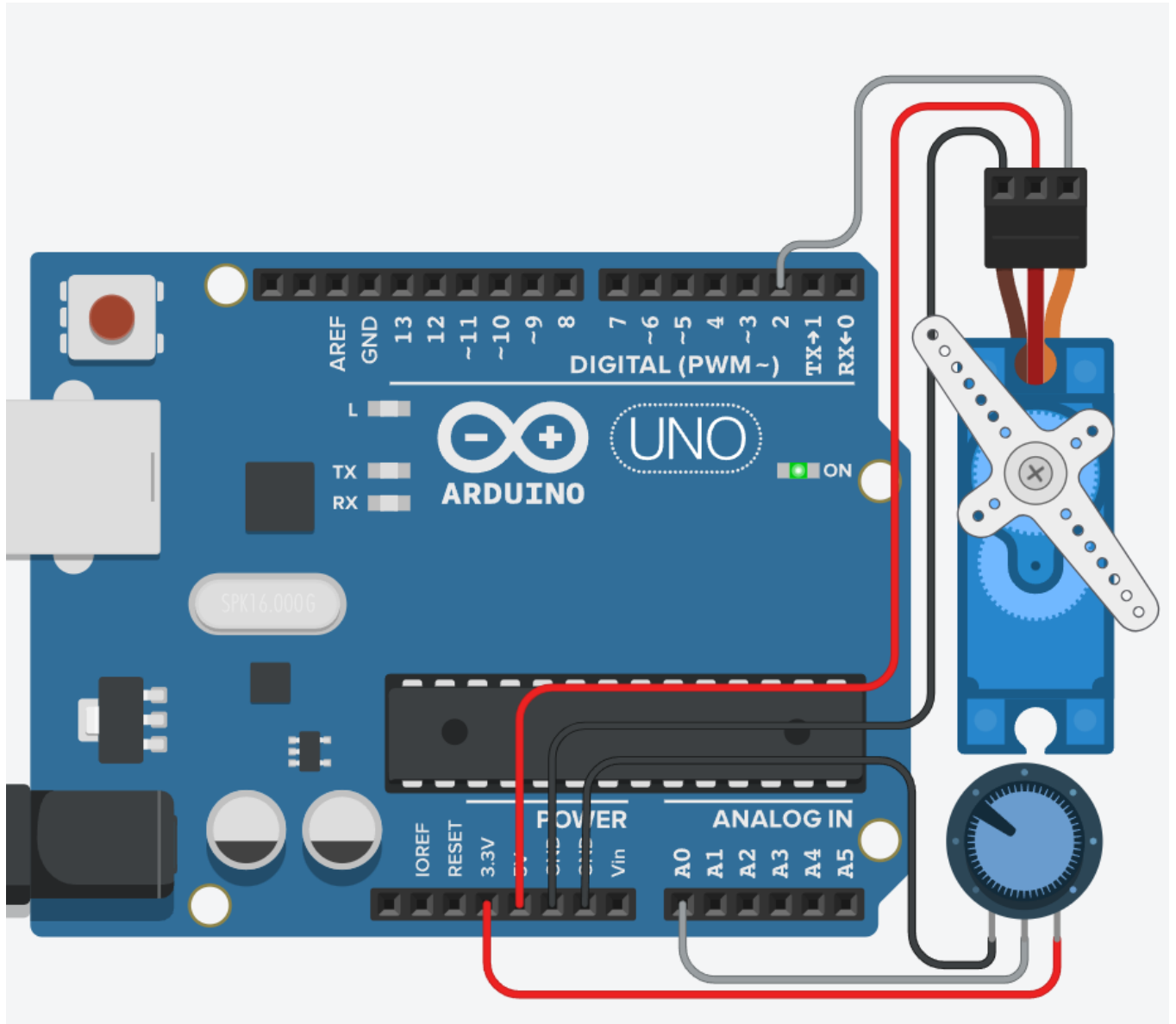


Рис.3 – Схема підключення сервоприводу та потенціометра до Arduino.

					5.123.1.28.07-ЛР				
Змін.	Арк.	№ докум.	Підпис	Дата	“Навчитися складати схеми підключення сервоприводу до ARDUINO та складати програму управління”.	Лім.	Арк.	Аркушів	
Розроб.		Молотков А.		27.05		у	1	3	
Перевір.		Дігтяр А.				ХДПК група 37-кі			
Т. контр.									
Н. Контр.									
Затверд.									

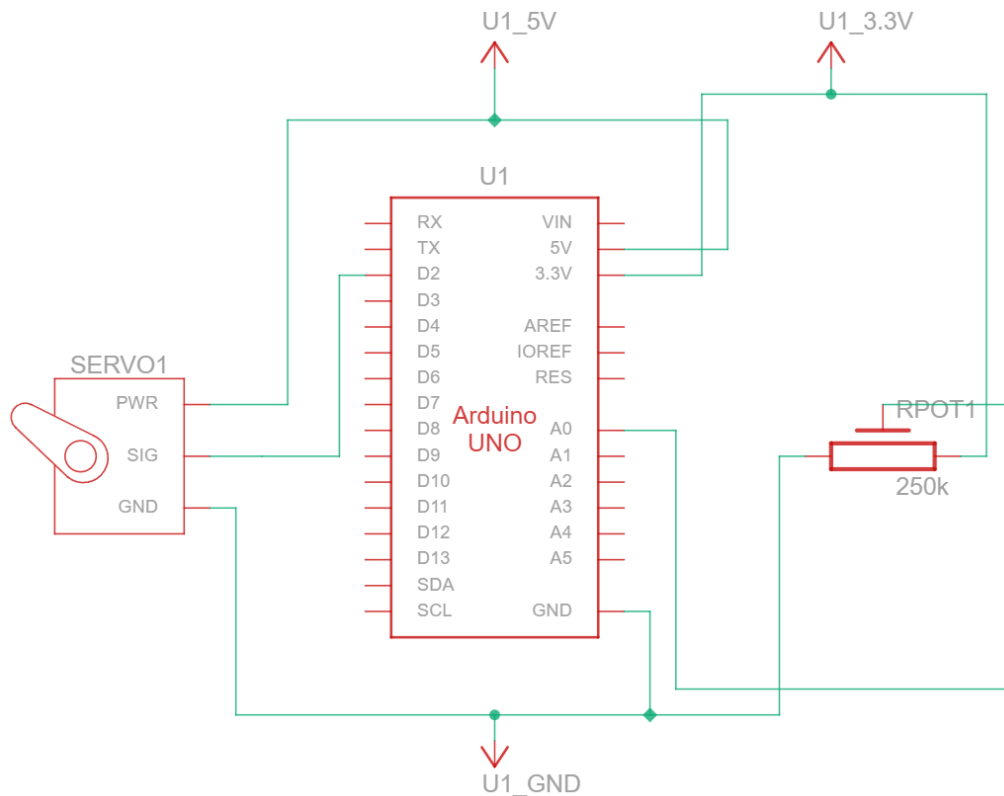


Рис.4 – Принципова електрична схема підключення сервоприводу та потенціометра до Arduino.

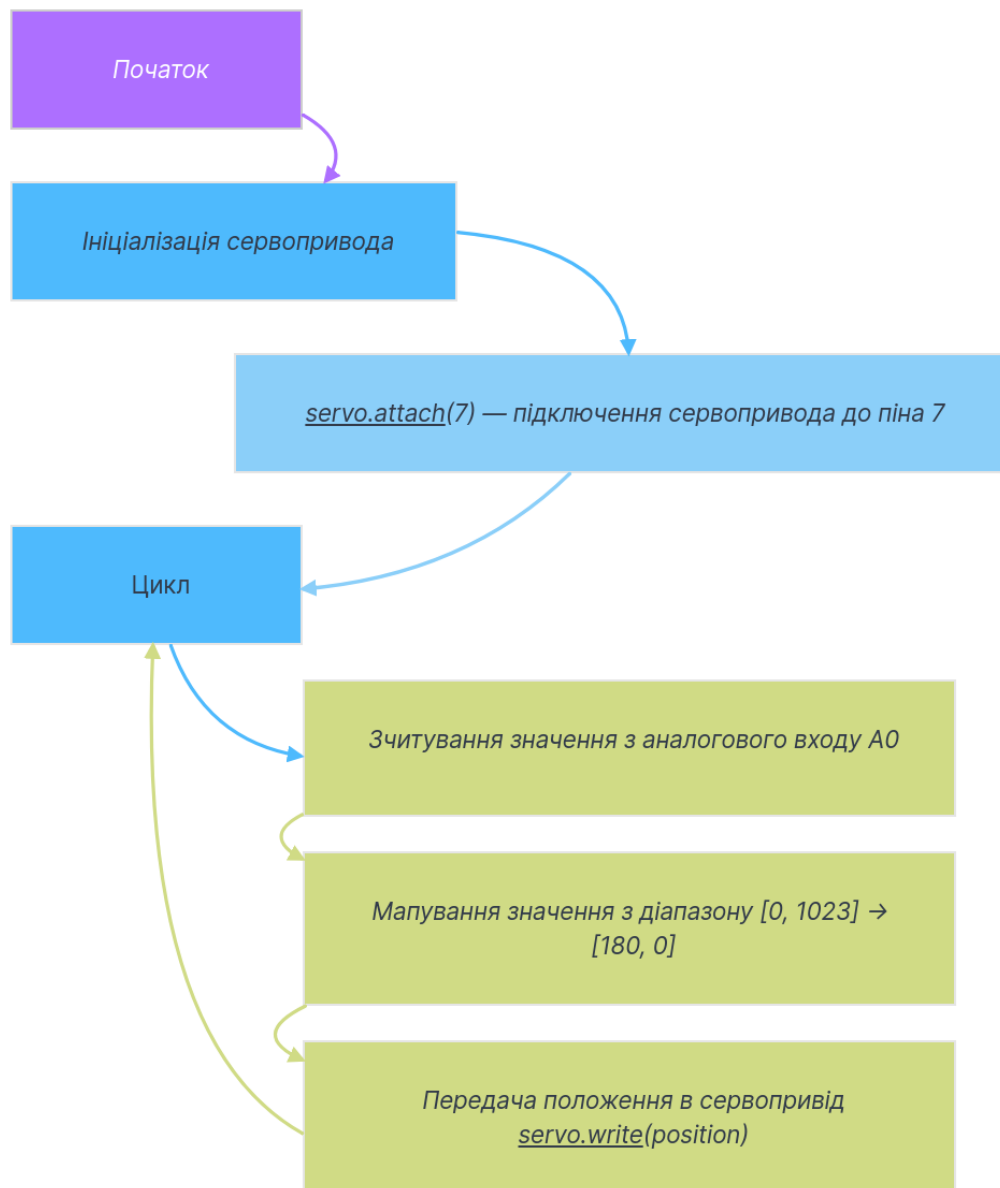
Скрипт 2:

```
#include <Servo.h>
Servo servo;
int position;
void setup()
{
  servo.attach(7);
}
void loop()
{
  int input = analogRead(A0); // Зчитуємо аналогового сигналу
  position = mapf(input, 0, 1023, 180, 0);
  //Мапування значення від 0 до 1024
  //на кут від 180 до 0 градусів
  servo.write(position); // Встановлюємо сервопривід у відповідне положення
}
float mapf(long x, long in_min, long in_max, long out_min, long out_max)
{
  return (float)(x - in_min) * (out_max - out_min) / (float)(in_max - in_min) + out_min;
  //Функція мапування
}
```

Контрольні запитання:

- 1) Зібрати схему та написати скетч в thinkercad. ✓
- 2) Алгоритм роботи лістингу № 2

					5.123.1.28.07-ЛР	Лист
Изм.	Лист	№ докум.	Подпись	Дата		2



- 3) Потенціометр — це змінний резистор, який дозволяє регулювати опір вручну. Потенціометр підключається трьома контактами: два крайні — до живлення (5V) і землі (GND), а середній — до аналогового входу Arduino (наприклад, A0).
- 4) Сервопривід складається з кількох основних частин, які забезпечують його точну і стабільну роботу.
 1. Електродвигун — перетворює електроенергію у механічний рух.
 2. Редуктор — знижує швидкість обертання двигуна та збільшує крутний момент.
 3. Контролер — обробляє сигнал керування і визначає, на який кут має повернутися вал.
 4. Вихідний вал — обертається відповідно до сигналу керування.
 5. Датчик положення — потенціометр, який визначає реальне положення валу і допомагає точно контролювати кут обертання.

Висновок: я досяг мети та вивчив мету ЛР, а також виконав інструкції та відповів на контрольні питання.

Лабораторна робота №8

Планування завдань у FreeRTOS

Мета: Ознайомитися з процесом створення та планування завдань у FreeRTOS, використовуючи Arduino.

Хід роботи :

Схема складає лише один мікроконтролер Arduino UNO.

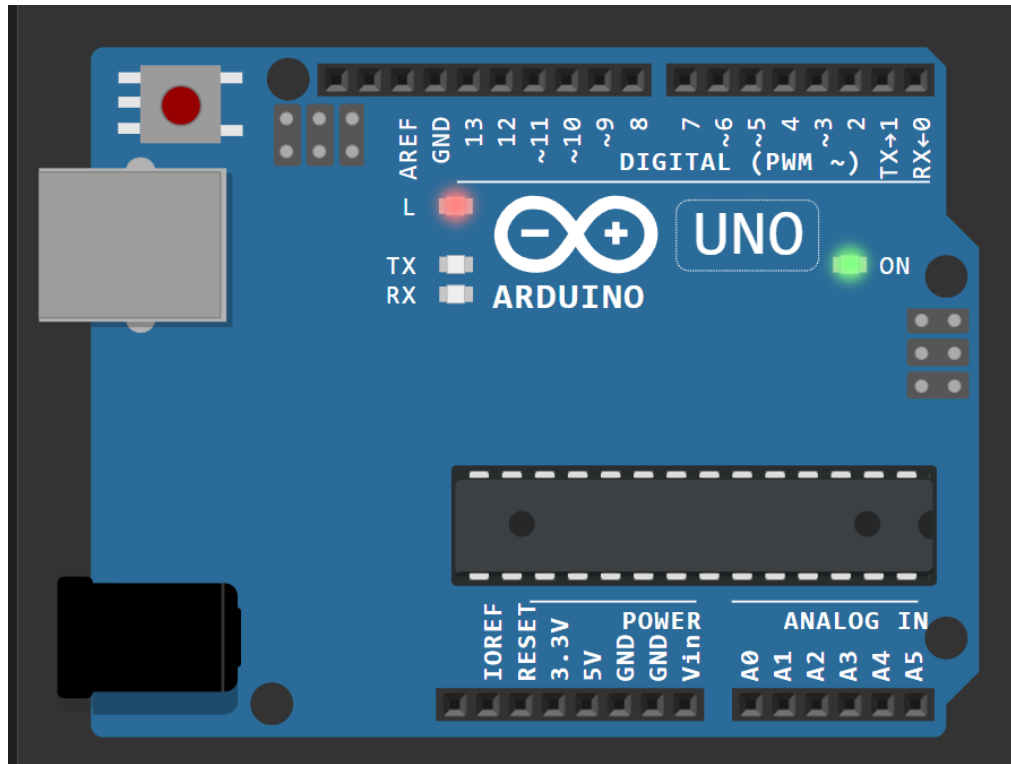


Рис.1 – Схема Arduino UNO зі плануванням завдань FreeRTOS.

Лістинг 1:

```
#include <Arduino_FreeRTOS.h>

void TaskBlink(void *pvParameters)
{
    pinMode(LED_BUILTIN, OUTPUT);
    for (;;)
    {
        digitalWrite(LED_BUILTIN, HIGH);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
        digitalWrite(LED_BUILTIN, LOW);
    }
}
```

					5.123.1.17.08-ЛР				
Змін.	Арк.	№ докум.	Підпис	Дата					
Розроб.		Молотков А.		27.05	“Ознайомитися з процесом створення та планування завдань у FreeRTOS, використовуючи Arduino.”.	Лім.	Арк.	Аркушів	
Перевір.		Дігтяр А.				у	1	4	
Т. контр.						ХДПК група 37-кі			
Н. Контр.									
Затверд.									

```

vTaskDelay(1000 / portTICK_PERIOD_MS);
}
}

void setup()
{
    // створення завдання
    xTaskCreate(TaskBlink, "Blink", 128, NULL, 1, NULL);
}

void loop()
{
    // порожня, бо FreeRTOS все бере на себе
}

```

Лістинг 2:

```

#include <Arduino_FreeRTOS.h>

void TaskBlink(void *pvParameters)
{
    pinMode(LED_BUILTIN, OUTPUT);
    for (;;)
    {
        digitalWrite(LED_BUILTIN, HIGH);
        vTaskDelay(500 / portTICK_PERIOD_MS);
        digitalWrite(LED_BUILTIN, LOW);
        vTaskDelay(500 / portTICK_PERIOD_MS);
    }
}

void setup()
{
    // створення завдання
    xTaskCreate(TaskBlink, "Blink", 128, NULL, 2, NULL);
    xTaskCreate(TaskBlink, "Blink", 128, NULL, 2, NULL); // вищий пріоритет
}

void loop() // порожня, бо FreeRTOS все бере на себе

```

					5.123.1.17.08-ЛР	Лист
						2
Изм.	Лист	№ докум.	Подпись	Дата		

Лістинг 3:

```
#include <Arduino_FreeRTOS.h>

void TaskBlink(void *pvParameters) {
    pinMode(LED_BUILTIN, OUTPUT);
    for (;;) {
        digitalWrite(LED_BUILTIN, HIGH);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
        digitalWrite(LED_BUILTIN, LOW);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void TaskBlink2(void *pvParameters) {
    pinMode(LED_BUILTIN, OUTPUT);
    for (;;) {
        digitalWrite(LED_BUILTIN, HIGH);
        vTaskDelay(500 / portTICK_PERIOD_MS);
        digitalWrite(LED_BUILTIN, LOW);
        vTaskDelay(500 / portTICK_PERIOD_MS);
        vTaskDelay(2000 / portTICK_PERIOD_MS); // додаткова пауза
    }
}

void setup() {
    xTaskCreate(TaskBlink, "Blink1", 128, NULL, 1, NULL);
    xTaskCreate(TaskBlink2, "Blink2", 128, NULL, 2, NULL);
}

void loop() {
    // FreeRTOS бере управління на себе
}
```

Спостерігаючи за роботою вбудованого світлодіода (LED_BUILTIN) при виконанні наданих кодів FreeRTOS, можна зробити наступні висновки:

1. У першому випадку, коли є лише одне завдання TaskBlink з пріоритетом 1, діод блимає стабільно з періодом 2 секунди. Це означає, що світлодіод світиться 1 секунду (HIGH) і не світиться 1 секунду (LOW). Оскільки немає інших завдань, що конкурують за ресурси, TaskBlink виконується безперешкодно, дотримуючись заданих затримок.

					5.123.1.17.08-ЛР	Лист
						3
Изм.	Лист	№ докум.	Подпись	Дата		

2. У цьому випадку обидва завдання TaskBlink одночасно намагаються керувати одним і тим же світлодіодом. Оскільки вони мають однаковий пріоритет і намагаються світити/вимикати діод з періодом 500 мс, виникає конфлікт. Візуально діод, ймовірно, буде дуже швидко і хаотично мерехтати або виглядатиме постійно увімкненим, оскільки планувальник FreeRTOS швидко перемикатиметься між завданнями, які борються за один і той же ресурс. Стабільного блимання з періодом 1 секунда не буде.
3. Поведінка діода буде визначатися завданням з вищим пріоритетом, тобто TaskBlink2 (пріоритет 2). Діод буде блимати з його характеристиками: світитися 500 мс, не світитися 500 мс, а потім матиме додаткову паузу в 2000 мс. Лише під час цієї довгої паузи TaskBlink2, планувальник передасть управління TaskBlink (пріоритет 1), і діод на короткий час почне блимати з періодом 2 секунди. Отже, загальна поведінка буде виглядати як швидке блимання з довгою паузою, під час якої іноді проглядається повільне блимання.

Висновок: я вивчив тему та досяг мети лабораторної роботи, а також відповів на контрольні питання. У ході виконання роботи, я зрозумів, що пріоритети у FreeRTOS є ключовим механізмом для керування виконанням завдань та розподілу ресурсів процесора. Завдання з вищим пріоритетом завжди отримує контроль над процесором, якщо воно готове, перериваючи завдання з нижчим пріоритетом, що забезпечує своєчасність критичних операцій. Функція `vTaskDelay()` дозволяє завданням добровільно звільнити процесор, даючи можливість іншим задачам отримати час виконання. Коли завдання змагаються за спільні ресурси з однаковими пріоритетами, можуть виникати конфлікти, що вимагає використання механізмів синхронізації.

Лабораторна робота 9

Таймери та черги FreeRTOS

Мета: Ознайомитися з процесом створення та використання черг у FreeRTOS, використовуючи Arduino.

Хід роботи :

Завдання: 1-3.3:

Схема:

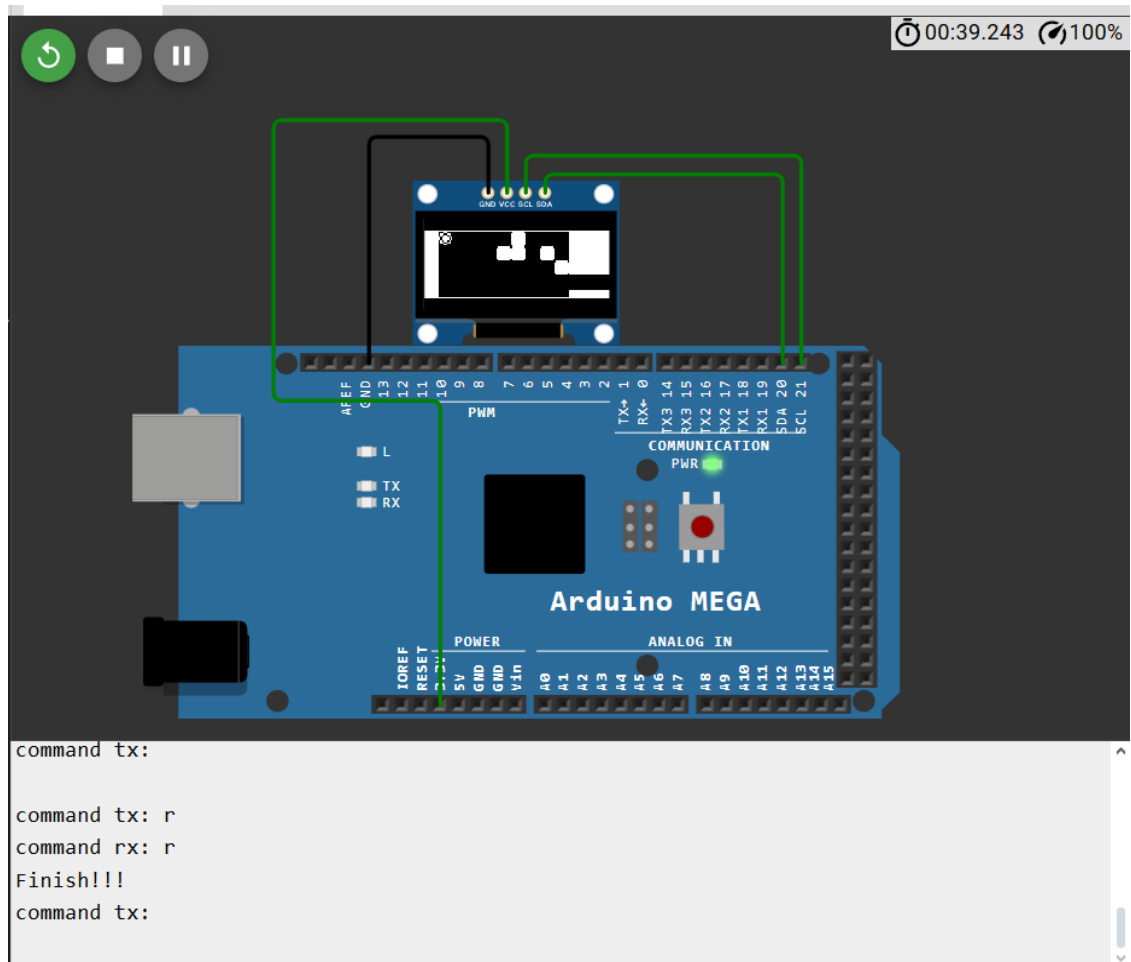


Рис.1 – Схема Arduino MEGA та SSD1306 OLED.

[Посилання](#)

Лістинг 1:

```
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Arduino_FreeRTOS.h>
#include <queue.h>
```

					5.123.1.17.09-ЛР			
Змін.	Арк.	№ докум.	Підпис	Дата	“Ознайомитися з процесом створення та використання черг у FreeRTOS, використовуючи Arduino”.	Лім.	Арк.	Аркушів
Розроб.		Молотков А.		05.06		у	1	4
Перевір.		Дігтяр А.				ХДПК група 37-кі		
Т. контр.								
Н. Контр.								
Затверд.								

```

#define OLED_RESET 4
Adafruit_SSD1306 display(128, 64, &Wire, OLED_RESET);

#define MOON_ROCK_COUNT 5
#define MOON_RUNNER_COUNT_COMMANDS 5

int moon_runner_x = 0;
int moon_runner_y = 0;
int moon_runner_position = 0;

#define MOON_RUNNER_H 5
#define MOON_RUNNER_W 10

#define POLE_MARGIN_LEFT 10
#define POLE_MARGIN_TOP 10

#define POLE_W 10
#define POLE_H 10

short moon_pole[37][2] = {
    {0,0}, {10,0}, {20,0}, {30,0}, {40,0}, {50,0}, {60,0}, {70,0},
    {80,0},
    {0,10}, {10,10}, {20,10}, {30,10}, {40,10}, {50,10}, {60,10},
    {70,10}, {80,10},
    {0,20}, {10,20}, {20,20}, {30,20}, {40,20}, {50,20}, {60,20},
    {70,20}, {80,20},
    {0,30}, {10,30}, {20,30}, {30,30}, {40,30}, {50,30}, {60,30},
    {70,30}, {80,30}, {90,30}
};

short moon_position_rock[MOON_ROCK_COUNT];

QueueHandle_t x_command_moon_rubber_queue;

// --- Прототипи ---
void runRightMoonRunner();
void runLeftMoonRunner();
void runUpMoonRunner();
void runDownMoonRunner();
void generateLevel();
int getXPosition(int position_x);
int getYPosition(int position_y);
bool checkCrash(short position_check);
bool checkFinish(short position_check);
void drawMoonRunner(short position);
void drawMoon();

// --- Отримання команд з UART ---
void getCommands(void *pvParameters){
    for(;;){
        if(Serial.available()){
            char command = Serial.read();
            Serial.print("command tx: ");
            Serial.println(command);
        }
    }
}

```

					5.123.1.17.09-ЛР	Лист
						2
Изм.	Лист	№ докум.	Подпись	Дата		

```

        if (command == 'r' || command == 'l' || command == 'u' || command
== 'd' || command == 'g') {
            xQueueSend(x_command_moon_rubber_queue, &command,
portMAX_DELAY);
        }
    }
    vTaskDelay(10 / portTICK_PERIOD_MS); // Невелика пауза
}

// --- Обробка команд по одній ---
void move(void *pvParameters){
    Serial.println("Start move");
    char command;
    for(;;){
        if (xQueueReceive(x_command_moon_rubber_queue, &command,
portMAX_DELAY) == pdPASS) {
            Serial.print("command rx: ");
            Serial.println(command);

            switch (command) {
                case 'r': runRightMoonRunner(); break;
                case 'l': runLeftMoonRunner(); break;
                case 'u': runUpMoonRunner(); break;
                case 'd': runDownMoonRunner(); break;
                case 'g': generateLevel(); break;
            }

            vTaskDelay(100 / portTICK_PERIOD_MS); // Затримка після обробки
        }
    }

void setup(){
    Serial.begin(9600);
    randomSeed(analogRead(0));

    x_command_moon_rubber_queue =
xQueueCreate(MOON_RUNNER_COUNT_COMMANDS, sizeof(char));
    if (x_command_moon_rubber_queue == NULL) {
        Serial.println("Failed to create queue.");
        while(1);
    }

    display.begin(SSD1306_SWITCHCAPVCC, 0x3c);
    display.clearDisplay();

    generateLevel();

    xTaskCreate(getCommands, "getCommands", 1000, NULL, 1, NULL);
    xTaskCreate(move, "move", 1000, NULL, 1, NULL);
}

```

```

void loop() {
    // FreeRTOS працює з task'ами, loop() не потрібен
}

void generateLevel() {
    if (x_command_moon_rubber_queue != NULL) {
        xQueueReset(x_command_moon_rubber_queue);
    }

    display.clearDisplay();
    moon_runner_position = 0;
    display.drawFastHLine(0, 9, 128, WHITE);
    display.drawFastHLine(0, 55, 128, WHITE);

    display.fillRect(0, 10, 10, 45, WHITE);
    display.fillRect(100, 10, 30, 30, WHITE);
    display.fillRect(100, 50, 30, 5, WHITE);

    drawMoon();
    drawMoonRunner(moon_runner_position);
}

void runRightMoonRunner() {
    drawMoonRunner(moon_runner_position + 1);
}

void runLeftMoonRunner() {
    drawMoonRunner(moon_runner_position - 1);
}

void runUpMoonRunner() {
    drawMoonRunner(moon_runner_position - 9);
}

void runDownMoonRunner() {
    drawMoonRunner(moon_runner_position + 9);
}

void drawMoonRunner(short position) {
    display.fillCircle(getXPosition(moon_runner_position)+4,
        getYPosition(moon_runner_position)+4, 2, BLACK);
    display.drawCircle(getXPosition(moon_runner_position)+4,
        getYPosition(moon_runner_position)+4, 4, BLACK);

    if(checkCrash(position)) {
        Serial.println("Crashed!!!");
        generateLevel();
        return;
    }

    if(checkFinish(position)) {
        Serial.println("Finish!!!");
        generateLevel();
        return;
    }
}

```

Изм.	Лист	№ докум.	Подпись	Дата

5.123.1.17.09-ЛР

Лист

4

```

    }

    display.fillCircle(getXPosition(position)+4,
getYPosition(position)+4, 2, WHITE);
    display.drawCircle(getXPosition(position)+4,
getYPosition(position)+4, 4, WHITE);
    display.display();

    moon_runner_position = position;
}

int getXPosition(int position_x){
    return moon_pole[position_x][0] + POLE_MARGIN_LEFT;
}

int getYPosition(int position_y){
    return moon_pole[position_y][1] + POLE_MARGIN_TOP;
}

void drawMoon(){
    for(short t = 0; t < MOON_ROCK_COUNT; t++){
        if (moon_position_rock[t] != 0) {
            display.fillRoundRect(getXPosition(moon_position_rock[t]),
getYPosition(moon_position_rock[t]), 10, 10, 2, BLACK);
        }
    }

    for(short t = 0; t < MOON_ROCK_COUNT; t++){
        short position_rock;
        do {
            position_rock = random(1, 34);
        } while (position_rock == moon_runner_position);

        moon_position_rock[t] = position_rock;
        display.fillRoundRect(getXPosition(position_rock),
getYPosition(position_rock), 10, 10, 2, WHITE);
    }
    display.display();
}

bool checkCrash(short position_check){
    for(short i = 0; i < MOON_ROCK_COUNT; i++){
        if(moon_position_rock[i] == position_check){
            return true;
        }
    }
    return false;
}

bool checkFinish(short position_check){
    return position_check == 36;
}

```

					5.123.1.17.09-ЛР	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

3.4 Покращення:

Посилання

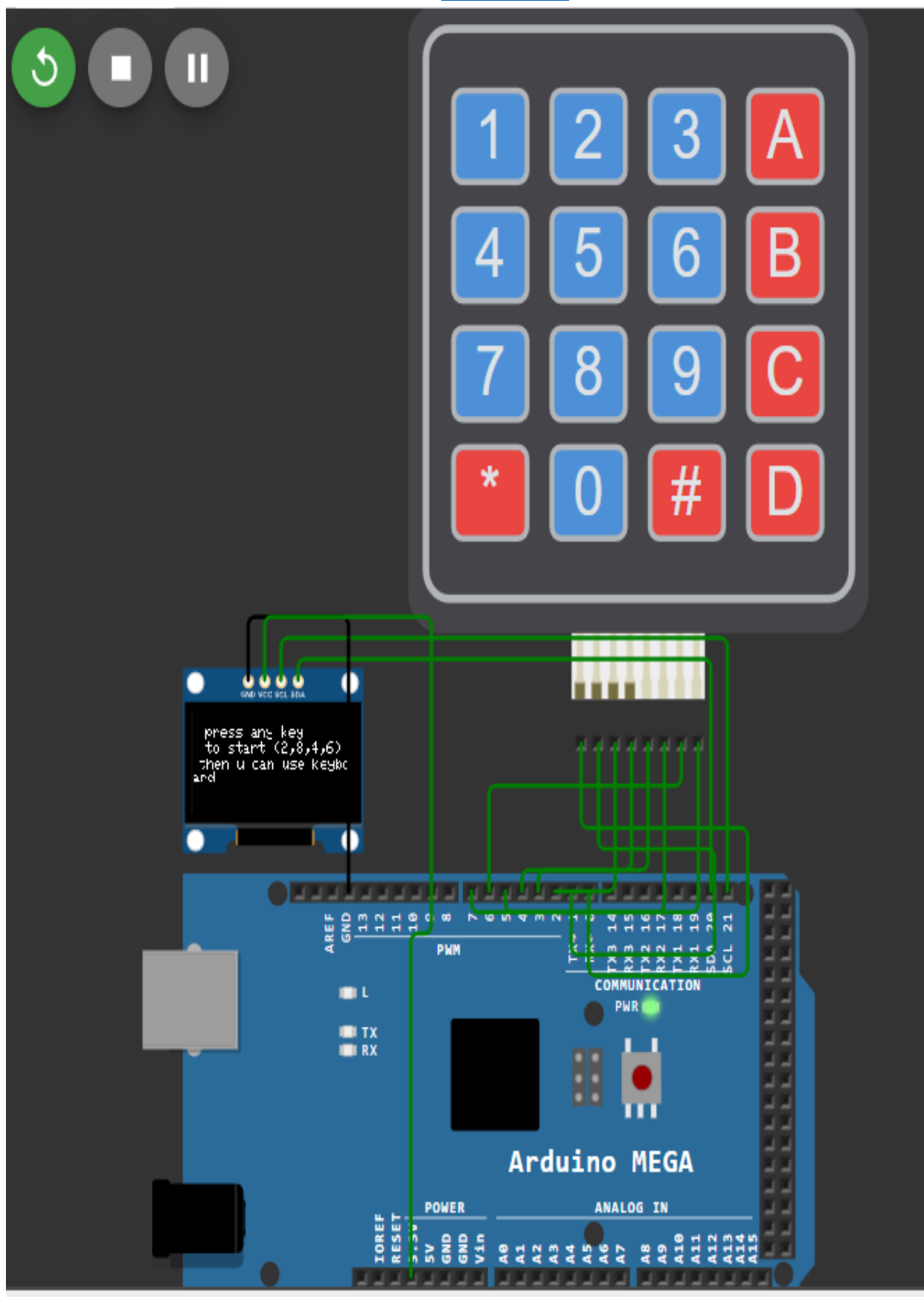


Рис.2 – Схема Arduino MEGA та SSD1306 OLED з кейпадом.

Изм.	Лист	№ докум.	Подпись	Дата

5.123.1.17.09-ЛР

Лист

6

Табл.1 — порівняння оригіналу та покращення

Характеристика	Система 1 (Код та зображення праворуч)	Система 2 (Код та зображення ліворуч)
Основна відмінність	Введення команд відбувається через послідовний порт (UART).	Введення команд відбувається через матричну клавіатуру (Keypad).
Зображення	Відображено лише плату Arduino MEGA та OLED-дисплей.	Додатково відображено матричну клавіатуру 4x4, підключену до Arduino.
Бібліотеки	Стандартні бібліотеки для OLED та FreeRTOS.	Додатково включено бібліотеку Keypad.h.
Введення користувача	Команди ('r', 'l', 'u', 'd', 'g') вводяться через послідовний монітор.	Команди вводяться з клавіатури (2-вгору, 8-вниз, 4-вліво, 6-вправо, *-новий рівень). Також підтримується введення через UART.
Початковий екран	Гра починається одразу після запуску та ініціалізації.	Перед стартом гри на OLED-дисплеї відображається інструкція "press any key to start (2,8,4,6) then u can use keyboard".
Логіка старту гри	Функція generateLevel() викликається безпосередньо в setup().	Функція generateLevel() викликається лише після отримання першої команди від користувача (через клавіатуру або UART). Введено флаг gameStarted.
Присутність Serial.println	Широке використання Serial.println для відладки та відображення отриманих команд (наприклад, "command tx:", "command rx:", "Crashed!!!", "Finish!!!").	Обмежене використання Serial.println для відладки (наприклад, "Queue creation failed!"). Основний вивід на дисплей.
Розпіновка клавіатури	Не застосовується.	Піни 0-3 використовуються для рядків клавіатури, 4-7 для стовпців.
Затримка в move	100 мс після обробки команди (vTaskDelay(100 / portTICK_PERIOD_MS)).	200 мс після обробки команди (vTaskDelay(200 / portTICK_PERIOD_MS)). Затримка 50 мс, коли гра не почалася.
Ігрова логіка	Ідентична (рух, перешкоди, фініш, перезапуск).	Ідентична (рух, перешкоди, фініш, перезапуск).
FreeRTOS	Обидві системи використовують FreeRTOS для багатозадачності (getCommands та move tasks).	Обидві системи використовують FreeRTOS для багатозадачності.

Покращена схема демонструє свою більшу інтерактивність та зручність для кінцевого користувача, оскільки дозволяє керувати грою за допомогою фізичної клавіатури, а також має дружнє стартове повідомлення.

Лістинг 2:

Лістинг 2:

```
#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Arduino_FreeRTOS.h>
#include <queue.h>
#include <Keypad.h>

#define OLED_RESET 4
Adafruit_SSD1306 display(128, 64, &Wire, OLED_RESET);

#define MOON_ROCK_COUNT 5
#define MOON_RUNNER_COUNT_COMMANDS 5

int moon_runner_position = 0;
#define POLE_MARGIN_LEFT 10
#define POLE_MARGIN_TOP 10

short moon_pole[37][2] = {
    {0,0},{10,0},{20,0},{30,0},{40,0},{50,0},{60,0},{70,0},{80,0},
    {0,10},{10,10},{20,10},{30,10},{40,10},{50,10},{60,10},{70,10},{80,10},
    {0,20},{10,20},{20,20},{30,20},{40,20},{50,20},{60,20},{70,20},{80,20},
    {0,30},{10,30},{20,30},{30,30},{40,30},{50,30},{60,30},{70,30},{80,30},
    {90,30}
};

short moon_position_rock[MOON_ROCK_COUNT];
QueueHandle_t x_command_moon_rubber_queue;

const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
    {'1','2','3','A'},
    {'4','5','6','B'},
    {'7','8','9','C'},
    {'*','0','#','D'}
};

byte rowPins[ROWS] = {0, 1, 2, 3}; // R1-R4 до пінів 0,1,2,3
byte colPins[COLS] = {4, 5, 6, 7}; // C1-C4 до пінів 4,5,6,7

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
```

					5.123.1.17.09-ЛР	Лист
						8
Изм.	Лист	№ докум.	Подпись	Дата		

```

volatile bool gameStarted = false; // Чи почалася гра

// Прототипи функцій
void generateLevel();
void runRightMoonRunner();
void runLeftMoonRunner();
void runUpMoonRunner();
void runDownMoonRunner();
void drawMoonRunner(short position);
int getXPosition(int position);
int getYPosition(int position);
bool checkCrash(short pos);
bool checkFinish(short pos);
void displayStartText();
void drawMoon();

void displayStartText() {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(WHITE);
    display.setCursor(10, 10);
    display.println("press any key ");
    display.setCursor(10, 20);
    display.println("to start (2,8,4,6)");
    display.setCursor(5, 30);
    display.println("then u can use keyboard");
    display.display();
}

void getCommands(void *pvParameters) {
    for (;;) {
        char command = '\0';

        if (Serial.available()) {
            char c = Serial.read();
            if (c == 'r' || c == 'l' || c == 'u' || c == 'd' || c == 'g') {
                command = c;
            }
        }

        char key = keypad.getKey();
        if (key) {
            switch (key) {
                case '2': command = 'u'; break; // Вгору
                case '4': command = 'l'; break; // Вліво
                case '6': command = 'r'; break; // Вправо
                case '8': command = 'd'; break; // Вниз
                case '*': command = 'g'; break; // Генерувати рівень
                default: break;
            }
        }

        if (command != '\0') {

```

```

        if (!gameStarted) {
            gameStarted = true;
            generateLevel(); // Стартуємо гру при першій команді
        }
        xQueueSend(x_command_moon_rubber_queue, &command, portMAX_DELAY);
    }

    vTaskDelay(20 / portTICK_PERIOD_MS);
}

}

void move(void *pvParameters) {
    char command;
    for(;;) {
        if (gameStarted) { // Команди виконуються лише якщо гра почалася
            if (xQueueReceive(x_command_moon_rubber_queue, &command,
portMAX_DELAY) == pdPASS) {
                switch (command) {
                    case 'r': runRightMoonRunner(); break;
                    case 'l': runLeftMoonRunner(); break;
                    case 'u': runUpMoonRunner(); break;
                    case 'd': runDownMoonRunner(); break;
                    case 'g': generateLevel(); break;
                }
                vTaskDelay(200 / portTICK_PERIOD_MS);
            }
        } else {
            vTaskDelay(50 / portTICK_PERIOD_MS); // Чекаємо старту гри
        }
    }
}

void setup() {
    Serial.begin(9600);
    randomSeed(analogRead(0));
    display.begin(SSD1306_SWITCHCAPVCC, 0x3c);
    display.clearDisplay();
    display.display();

    displayStartText(); // Показуємо текст перед стартом

    x_command_moon_rubber_queue =
xQueueCreate(MOON_RUNNER_COUNT_COMMANDS, sizeof(char));
    if (x_command_moon_rubber_queue == NULL) {
        Serial.println("Queue creation failed!");
        while(1);
    }

    // НЕ викликаємо generateLevel() одразу, чекаємо на команду для
    старту

    xTaskCreate(getCommands, "GetCmd", 1000, NULL, 1, NULL);
    xTaskCreate(move, "Move", 1000, NULL, 1, NULL);
}

```

```

void loop() {
    // FreeRTOS керує виконанням
}

void generateLevel() {
    xQueueReset(x_command_moon_rubber_queue);
    display.clearDisplay();
    moon_runner_position = 0;

    display.drawFastHLine(0, 9, 128, WHITE);
    display.drawFastHLine(0, 55, 128, WHITE);

    display.fillRect(0, 10, 10, 45, WHITE);
    display.fillRect(100, 10, 30, 30, WHITE);
    display.fillRect(100, 50, 30, 5, WHITE);

    drawMoon();

    drawMoonRunner(moon_runner_position);
}

void drawMoon() {
    for (short i = 0; i < MOON_ROCK_COUNT; i++) {
        if (moon_position_rock[i] != 0)
            display.fillRoundRect(getXPosition(moon_position_rock[i]),
getYPosition(moon_position_rock[i]), 10, 10, 2, BLACK);
    }

    for (short i = 0; i < MOON_ROCK_COUNT; i++) {
        short position;
        do {
            position = random(1, 34);
        } while (position == moon_runner_position);

        moon_position_rock[i] = position;
        display.fillRoundRect(getXPosition(position),
getYPosition(position), 10, 10, 2, WHITE);
    }
    display.display();
}

void drawMoonRunner(short position) {
    // Очистити попередні положення
    display.fillCircle(getXPosition(moon_runner_position)+4,
getYPosition(moon_runner_position)+4, 2, BLACK);
    display.drawCircle(getXPosition(moon_runner_position)+4,
getYPosition(moon_runner_position)+4, 4, BLACK);

    if (checkCrash(position) || position < 0 || position > 36) {
        generateLevel();
        return;
    }
}

```

```

    if (checkFinish(position)) {
        generateLevel();
        return;
    }

    // Малюємо гравця в новій позиції
    display.fillCircle(getXPosition(position)+4,
getYPosition(position)+4, 2, WHITE);
    display.drawCircle(getXPosition(position)+4,
getYPosition(position)+4, 4, WHITE);
    display.display();

    moon_runner_position = position;
}

void runRightMoonRunner() {
    drawMoonRunner(moon_runner_position + 1);
}

void runLeftMoonRunner() {
    drawMoonRunner(moon_runner_position - 1);
}

void runUpMoonRunner() {
    drawMoonRunner(moon_runner_position - 9);
}

void runDownMoonRunner() {
    drawMoonRunner(moon_runner_position + 9);
}

int getXPosition(int position) {
    return moon_pole[position][0] + POLE_MARGIN_LEFT;
}

int getYPosition(int position) {
    return moon_pole[position][1] + POLE_MARGIN_TOP;
}

bool checkCrash(short pos) {
    for (short i = 0; i < MOON_ROCK_COUNT; i++)
        if (moon_position_rock[i] == pos) return true;
    return false;
}

bool checkFinish(short pos) {
    return pos == 36;
}

```

4. Опис, як у мені у роботі допомогли черги.

У цій грі, черги були дуже корисними, тому що вони дозволили мені обробляти команди гравця послідовно. Коли гравець вводить кілька команд для космічного корабля (наприклад, "вправо", "вгору", "вправо"), ці команди

					5.123.1.17.09-ЛР	Лист
Изм.	Лист	№ докум.	Подпись	Дата		12

не виконувалися миттєво. Замість цього, вони ставилися в чергу. Це схоже на те, як люди стоять у черзі в магазині – кожен обслуговується по черзі. Таким чином, дві окремі "частини" моєї програми могли працювати ефективно: одна "частина" (функція `getCommands`) збирала всі команди, які надходили від гравця, і додавала їх у чергу. Інша "частина" (функція `move`) брала ці команди з черги по одній і виконувала їх. Це дозволило уникнути хаосу, коли корабель міг би отримати одразу багато команд і не знати, яку виконувати першою, або ж пропустити деякі. Завдяки чергам, рухи корабля були плавними та передбачуваними, що зробило гру кращою.

Висновок: я вивчив тему та досяг мети лабораторної роботи, виконав усі завдання ходу роботи.

					5.123.1.17.09-ЛР	Лист
						13
Изм.	Лист	№ докум.	Подпись	Дата		